PHYSICAL AI · RESEARCH SEMINAR

HarnessPAI

Beyond the Action Model

An Evolving Harness for Physical AI

How can executable, evolving code
unlock the capabilities an action model already has?

Darwin Agent Team
Research talk & methodology discussion

Code as Harness | Dual Timescales | Skill Evolution

Based on the manuscript and supplied project assets · Research seminar · 27 main slides / 3 appendix slides

RESEARCH SEMINAR

Roadmap: from capability gaps to an evolving system

01 **Background: an action model is not a complete agent**
Why do visual grounding and instruction following become bottlenecks?
03—07

02 **Motivation: use existing motor skills in the right way**
What should a harness add while the action model stays frozen?
08—10

03 **Method: code as a harness, evolving on two timescales**
Why stable execution? How do we verify, repair, and reuse programs?
11—22

Then Where do the gains come from, what transfers, and what remains unsolved? Evidence & takeaways · 23—27

Talk structure · Introduction / Motivation / Framework / Experiments

01 · BACKGROUND & OBSERVATIONS

Physical AI needs more than action generation

A complete physical agent must perceive, understand, and act.

V

Perceive & ground

Where are the objects?
What has changed?

Current scene

→

L

Understand & reason

Which object is intended?
What constraints must hold?

Task semantics

→

A

Act & control

How should the robot move?
How is contact handled?

Physical interaction

VLA · Vision-Language-Action

Maps visual observations and instructions to actions.

WAM · World Action Model

Also models how the world evolves with actions.

The gap Action-centric training and evaluation do not automatically ensure reliable grounding or instruction following.

Manuscript §1 Introduction; §3.1 Why Do VLA/WAM Models Need a Harness?

01 · BACKGROUND & OBSERVATIONS

Near-perfect standard scores can hide brittle behavior

SAME ACTION BACKBONE · $\pi_{0.5}$

High success is not robust understanding.

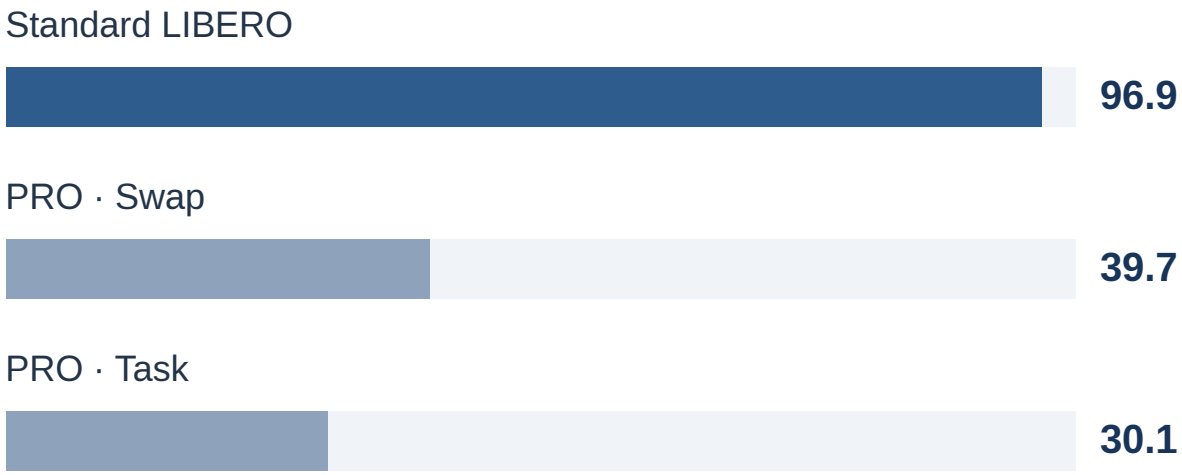
LIBERO-PRO perturbs familiar tasks to test whether a policy can re-ground its actions when layouts or instructions change.

Swap · scene perturbation
Exchange initial object positions.

Task · instruction perturbation
Change the instruction and intended target.

Task success

Higher is better · %



Aggregated over Spatial, Object, and Goal suites.
A standard-versus-perturbed comparison, not a causal training ablation.

Manuscript §5.2 / Table 4 / LIBERO-PRO results · The paper's own evaluations of $\pi_{0.5}$

01 · BACKGROUND & OBSERVATIONS

Language shift: one “not” changes which bowl to grasp

OBSERVATION 01

Pick up the black bowl

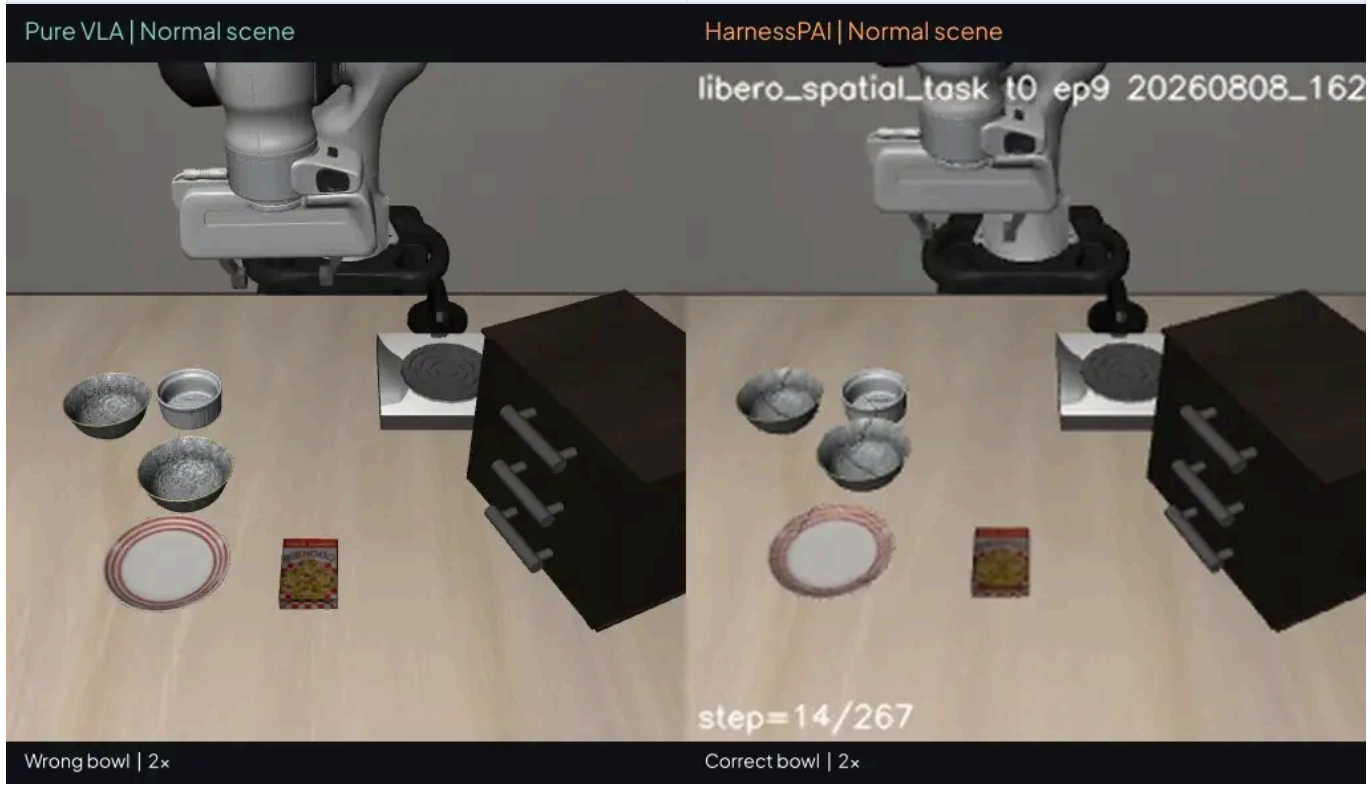
not between

the plate and the ramekin.

- The added negation changes the target.
- The VLA still grasps the original bowl.
- The harness re-grounds the target before invoking motor skills.

> Grasping a bowl is not the same as **grasping the right bowl.**

Pure VLA · wrong target	HarnessPAI · correct target



Selected rollouts illustrate behavior; success rates are not inferred from this pair.
The manuscript also shows a masking intervention that does not resolve this case.

Manuscript §3.1 / Fig. 4; project clip: motivation-language · Different selected episodes; 2× source playback

01 · BACKGROUND & OBSERVATIONS

Scene shift: the motion works, but the destination is stale

OBSERVATION 02

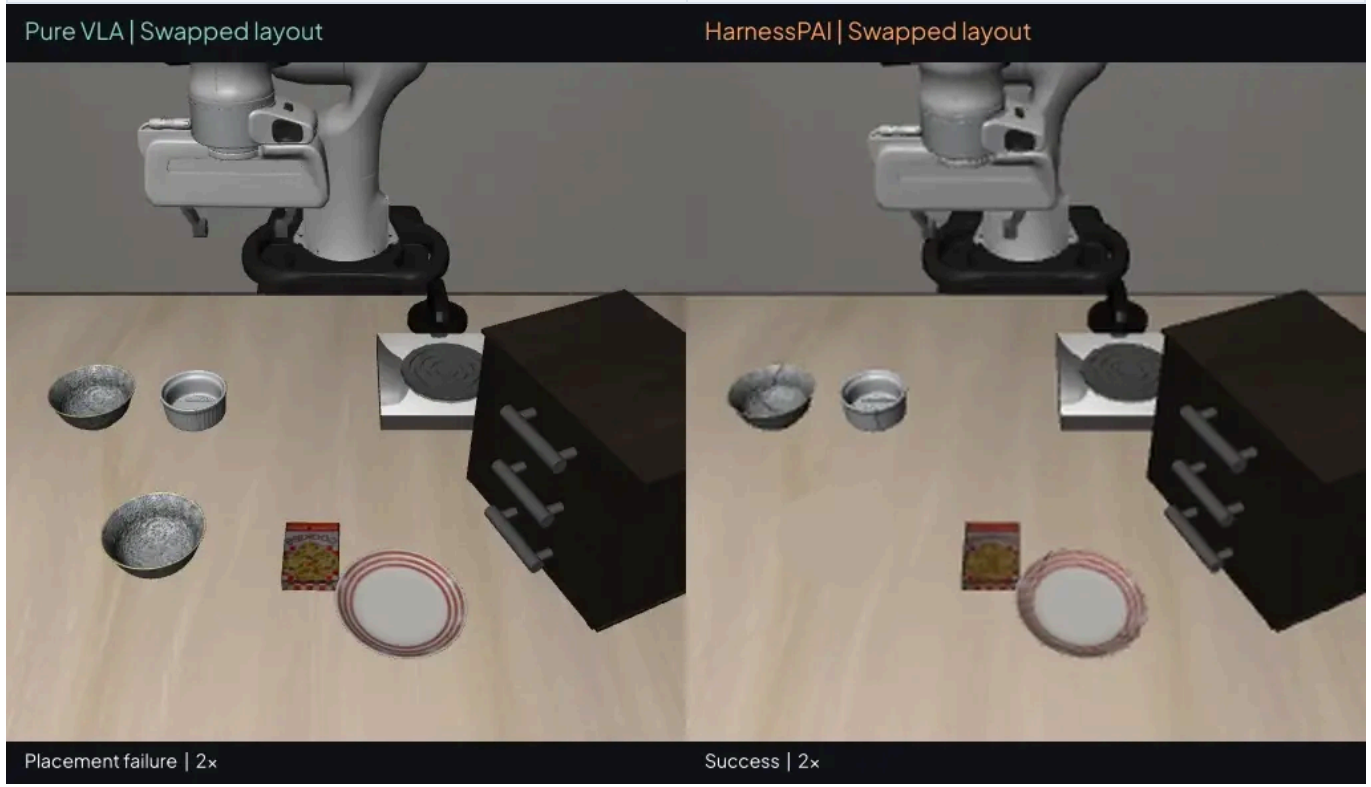
Same instruction.

A new destination.

- Grasping and transport still happen.
- The VLA moves toward the plate’s old position.
- The harness re-locates the destination in the current scene.

> Update **where** to act,
> not just relearn **how** to move.

Pure VLA · placement failure	HarnessPAI · re-grounded goal



This clip swaps the plate and a distractor bowl; the paper’s static example swaps the plate and ramekin. They illustrate the same failure class, not the same rollout.

Manuscript §3.1 / Fig. 5; project clip: motivation-visual-swap · 2× source playback; initial-state equality unverified

01 · BACKGROUND & OBSERVATIONS

The missing pieces: grounding, semantics, and verification

01 **The wrong object**

A valid motion, but the wrong referent.

02 **The wrong destination**

Grasp and transport succeed; grounding does not.

03 **Near the button is not on the button**

Reaching the vicinity does not complete the press.

WAM · unsuccessful press	HarnessPAI · completed task



RoboCasa · microwave button task · head-camera view

Careful attribution These examples motivate system-level support; they do not prove training-induced V/L loss or intrinsic imprecision of generative action heads.

Manuscript §3.1 / Fig. 6; project clip: motivation-microwave-column2 · Illustrative rollouts do not isolate a unique cause

02 · MOTIVATION & DESIGN CHOICE

Motivation: use existing motor competence in the right way

RESEARCH QUESTION

If the model can still reach, grasp, and transport,
how do we make it act correctly after a perturbation?

Re-ground the goal

Resolve the instruction
in the current scene

→

Invoke existing skills

Retain the frozen model's
atomic motor competence

→

Verify before advancing

Check grasp and placement;
gate the next stage

Reliable behavior = Motor competence + Explicit organization

Precondition: useful primitives already exist. A harness cannot create a capability absent from both the model and available control APIs.

Manuscript §3.3 Why Use Code as a Physical AI Harness?; §6.2 When the Framework Works

02 · MOTIVATION & DESIGN CHOICE

Harness design: why not an online agent or pure code?

Design	Who decides next?	Learned motor skills	Main trade-off
Pure VLA / WAM	Action model	Retained	Grounding and checks often implicit
Online reasoning harness	High-level agent	Retained	Repeated online deliberation
Code as Policy	Program + control APIs	Not necessarily retained	Contact-rich control depends on APIs
Code as Harness	**Validated task program**	**Replaceable primitives**	**Upfront evolution; repeated reuse**

Do not replace the action model

Code organizes grounding and checks.
The model supplies learned manipulation.

Do not rethink every step

Reuse an executable strategy.
Move program revision between rollouts.

Manuscript §1, §2 Related Work, §3.3 · Conceptual design comparison, not a controlled ablation

02 · MOTIVATION & DESIGN CHOICE

Why code? It turns a strategy into executable knowledge

01

Executable

RUN THE STRATEGY

Connect perception, action calls, and branches—not just a plan in words.

```
observe → act → check
```

02

Inspectable

LOCATE THE FAILURE

Make object selection and stage transitions readable and editable.

```
if not grasped: stop
```

03

Evolvable

REPAIR AND REUSE

Map failures to nodes, revise their implementation, and re-validate.

```
failure → patch → test
```

Deployment logic Especially useful when a task is repeated: invest in getting a program right, then reuse it.

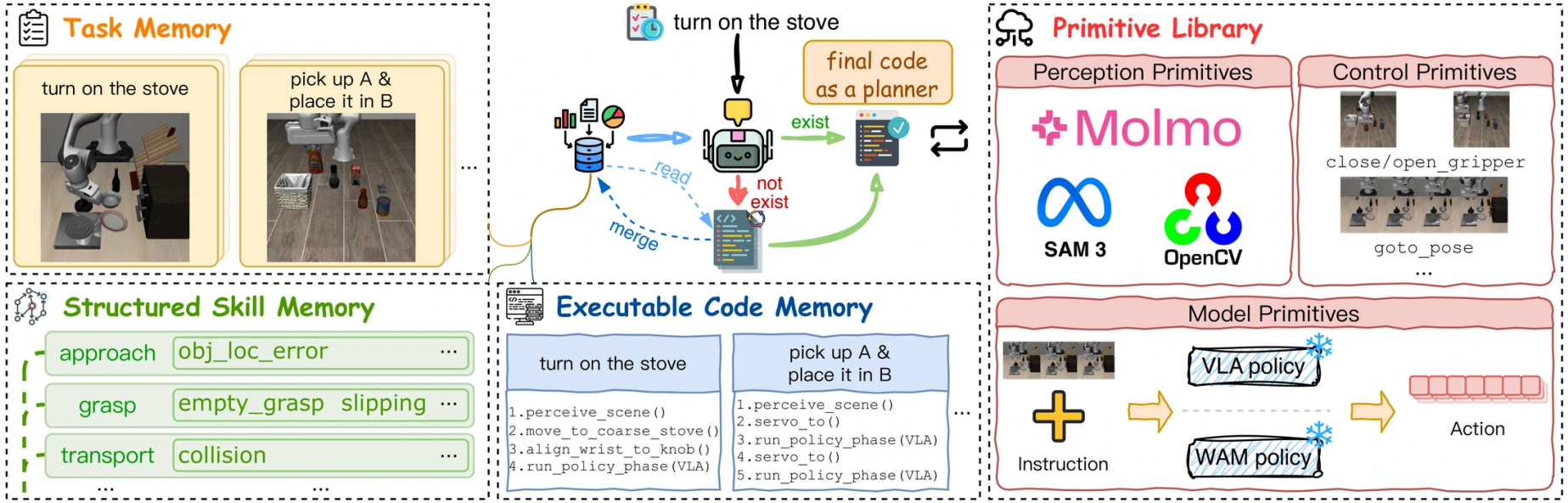
Reuse programs and repair knowledge—not an assumption of unlimited physical resets or guaranteed zero-shot success on every new task.

Manuscript §3.3 · Executable / inspectable / evolvable · Repeated deployment is the primary target regime

03 · FRAMEWORK & EVOLUTION

Architecture: code connects memory, perception, and action

The action model is one **primitive** in the system—not the whole agent.



Task Memory · Structured Skill Memory · Executable Code Memory · Primitive Library

Task memory

Is a reusable program already available?

Skill + code memories

How to repair; where the implementation lives.

Primitive library

Perception, control, VLA / WAM / PPO.

Manuscript §4.1 / Fig. 8 · Original framework figure from the supplied project assets

03 · FRAMEWORK & EVOLUTION

What does it mean to wrap an action model in code?

task_program.py — Illustrative pseudocode

```
def execute(instruction, robot, action_model):
    scene = perceive(robot.observe())
    target = ground(instruction, scene)
    destination = locate_destination(scene)

    robot.approach(target.pose)
    action_model.grasp(target)
    if not verify_grasp(robot.observe()):
        return safe_stop()

    robot.transport(destination)
    action_model.place(destination)
    return verify_goal(robot.observe())
```

GROUND

Explicit objects and poses

Resolve the task in the current scene.

DELEGATE

Retain learned actions

Freeze weights; do not hand-code all contact.

VERIFY

Gate execution

No verified grasp, no blind transition to transport.

Teaching pseudocode based on manuscript §4.1 · Not a real project API or deployable robot-control implementation

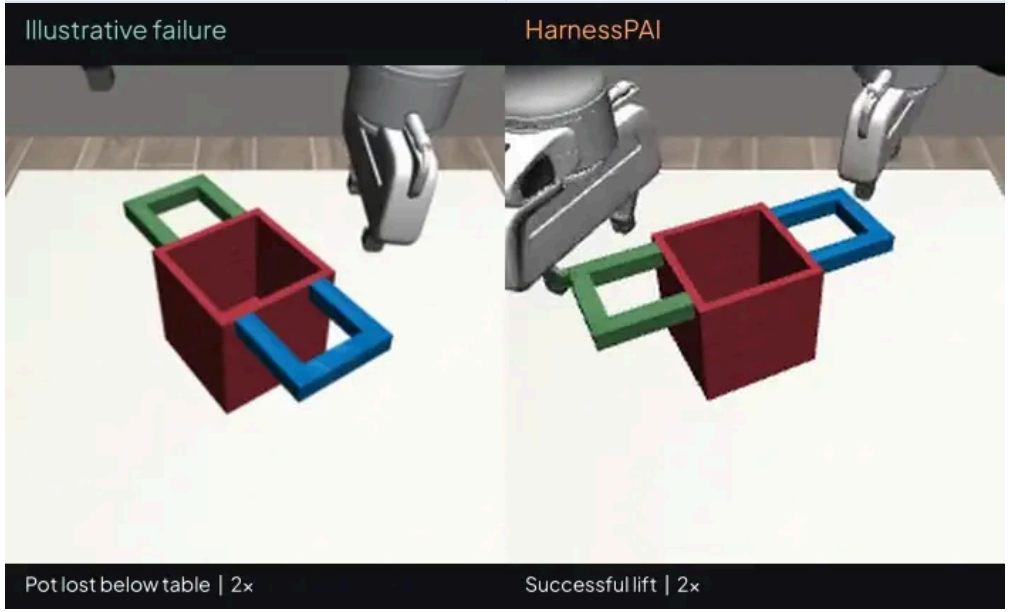
03 · FRAMEWORK & EVOLUTION

Why open-loop execution? Some failures cannot be undone

PHYSICAL ACTIONS CHANGE THE WORLD

There is no reliable “undo” button.

- A bad grasp can drop or overturn an object.
- After the scene changes, recovery may be impossible within the rollout.
- Reasoning after failure may already be too late.



Left: a scripted drop; the pot is lost below the table. Right: successful lift.
This is an illustration, not a pure-WAM baseline or a RoboCasa comparison.

Program-level open-loop Run validated logic with pre-action checks. Keep sensor feedback; move code revision between rollouts, before the next risky execution.

Manuscript §3.2; supplied irreversible-comparison.mp4 · robosuite TwoArmLift illustration; 2× source playback

03 · FRAMEWORK & EVOLUTION

Prevent errors now; improve the program before the next run

A physical mistake cannot always be fixed by undoing a line of code.

WRONG DECISION
Bad grasp / collision

→

PERSISTENT CHANGE
Object falls or spills

→

RECOVERY COST
The rollout may be lost

During this run: prevent the error

- Ground targets and geometric constraints.
- Gate progress with predefined checks.
- Recover or stop within defined policies.

Before the next: repair the program

- Locate weak nodes from execution evidence.
- Validate fixes in resettable experiments.
- Deploy only a validated candidate.

Objective Improve first-attempt reliability, rather than rely on unlimited physical retries.

Manuscript §3.2 Why Open-Loop Execution? · Conceptual state-transition illustration

03 · FRAMEWORK & EVOLUTION

Two timescales: execute a fixed program; evolve between runs

TIMESCALE ① · WITHIN

One rollout

Program P_k is fixed

Observe → Ground → Act → Check

↖ Read current observations; follow predefined branches, recovery, or stopping rules ↙

No online program rewriting. Perception and low-level feedback remain active.

TIMESCALE ② · ACROSS

Between rollouts

$P_k \rightarrow P_{k+1}$

Evidence → Diagnose → Revise → Validate

↖ Repair failures; promote a candidate only after it passes the success contract ↙

Code and repair knowledge evolve; action-model parameters θ stay frozen.

Key distinction Program-level open-loop ≠ blind action replay. Cross-rollout evolution ≠ continuous online LLM replanning.

Manuscript §3.2, §4.1–4.3 · “Open-loop” refers to the program, not sensors or low-level feedback control

03 · FRAMEWORK & EVOLUTION

Execution: checks gate the next step, not just the final score

Perception + geometry
Ground
Targets and poses

→

Control code
Approach
Reach the object

→

Learned model
Grasp
Establish contact

→

Control code
Transport
Move geometrically

→

Learned model
Place
Complete placement

✓
CHECK

Transport only after verify_grasp succeeds

Failed check → predefined recovery or stopping.
Not an invitation for the high-level LLM to rewrite the program mid-run.

What code contributes

Explicit constraints, geometry, control, and checks.

What the action model contributes

Learned manipulation where atomic skills are useful.

Manuscript §4.1; §4.3 / Table 3 hybrid rollout · Stage allocation is illustrative and depends on task and capability

03 · FRAMEWORK & EVOLUTION

Before evolution: define the task and build an editable skeleton

Task description | Environment & interfaces | Success contract C | Budget T

OPTIONAL 01

Extract a task graph

Recover stage order from successful video or a related task.

`approach → grasp → place`

OPTIONAL 02

Assess model capability

Observe each node and decide where code should help.

`good / needs code / mixed`

OPTIONAL 03

Initialize perception

Organize SAM / Molmo and other primitives as the task needs.

`scene → grounded targets`

↓

Candidate program P_0

With a graph: modularize by node. Without one: a single-file program can suffice.

A full human expert trajectory is not required. The graph supplies stage structure; every configuration enters the same execution–diagnosis–repair loop.

Manuscript §4.2 Closed-Loop Evolution · All three initialization stages below are optional

03 · FRAMEWORK & EVOLUTION

Evolution: writing code is only the start of validation

01

Generate / repair

Edit an isolated candidate;
protect validated programs.

02

Pre-flight

Check interfaces and execution;
separate bugs from policy failures.

03

Targeted probe

Recheck 3 previously failing seeds;
test the intended repair first.

04

Full regression batch

Run every contracted suite / episode;
look for new and recurring failures.

05

Diagnose + gate

Use node-wise video diagnosis;
apply authoritative success criteria.

06

Synthesize the next step

Failed: revise again.
Passed: promote code; distill skills.

↻ Failure → retrieve relevant skills → return to 01

✓ Full pass → Promote + Distill

Stop when all contracted development rollouts pass or the budget is exhausted. Failed candidates retain diagnostics but do not enter the verified code library.

Manuscript §4.3 / Algorithm 1 · Probe the previous round's 3 failing seeds, then run the full development batch

03 · FRAMEWORK & EVOLUTION

Who decides success? The code writer should not grade itself

SUCCESS CONTRACT · FIXED BEFORE EVOLUTION

C = scope + authoritative goal + terminal state + non-success signals

The writer, diagnoser, and reviewer all receive the same contract.

Code writer

Repairs the candidate and synthesizes the next revision.

Proposes a solution

Independent video diagnoser

Explains where failures occur relative to workflow nodes.

Diagnostic evidence, not a guarantee

Deterministic gate

Checks the authoritative goal over the contracted scope.

Controls promotion

Clean exit ≠ Program terminal state ≠ High dense reward ≠ **Task success**

Manuscript §4.3 · Success contract, independent video diagnosis, deterministic gate, and candidate isolation

03 · FRAMEWORK & EVOLUTION

Turn one failure into a verifiable repair at a specific node

EXAMPLE · PLACE NODE

Observed failure: a grasped object moves toward the plate’s old position.

LOCALIZE

P_0 · stale destination

```
destination = stored_pose
place(destination)
```

Locate the problem in grounding and placement—not “the action model is bad.”

REPAIR

P_1 · re-ground the scene

```
scene = observe()
destination = locate_plate(scene)
```

Add perception and geometry while retaining useful action primitives.

VERIFY & RETAIN

P_2 · check the final relation

```
place(destination)
verify_object_on_plate()
```

Recheck failures, run full regression, then promote only if validated.

Skill record Problem: stale goal → Repair: re-ground + verify placement → Reference: maintained code.

Teaching example based on manuscript §4.3–4.4 · Not an actual commit history; no invented success rates

03 · FRAMEWORK & EVOLUTION

Remember both how to execute and how to repair

Code library · executable knowledge

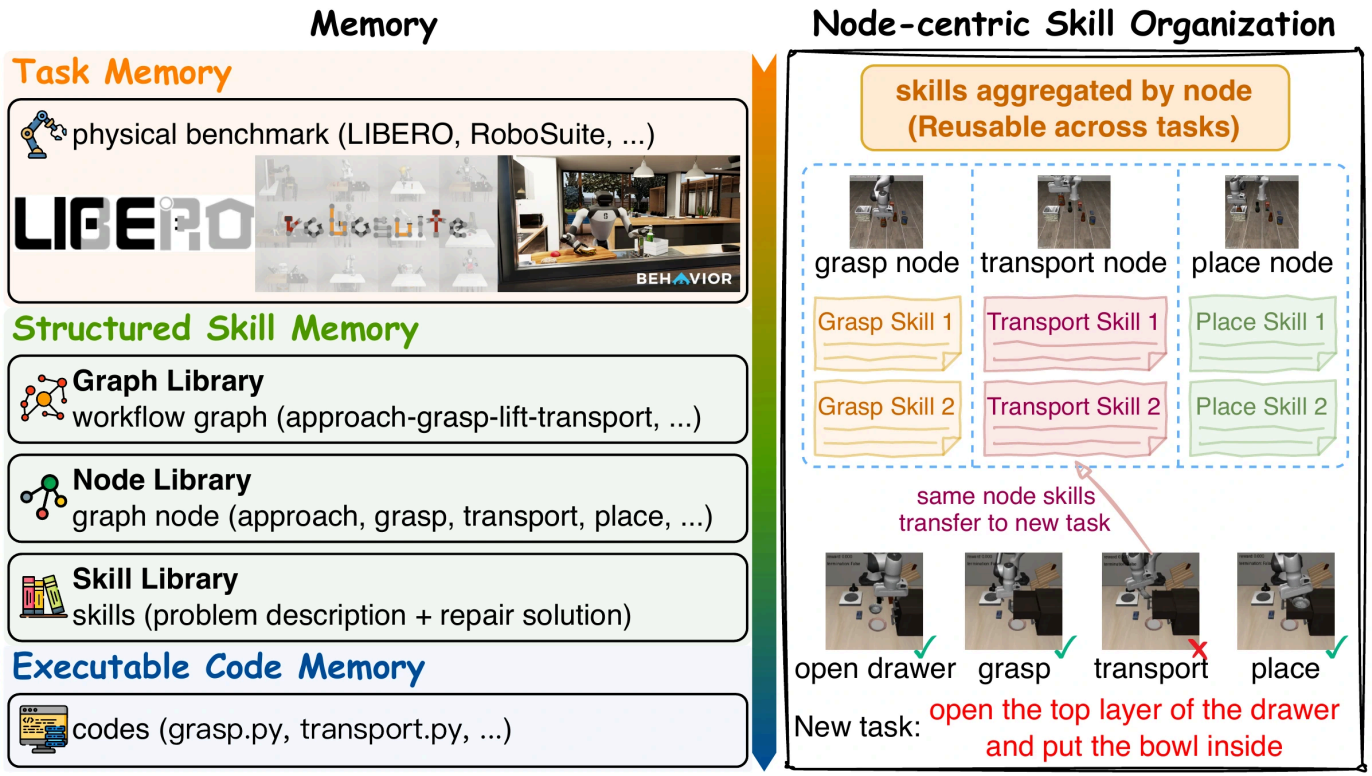
Validated task programs, control logic, and maintained repair implementations.

Skill library · repair experience

Problem → Repair → Code reference

Index skills by grasp / transport / place nodes, not just a flat collection of notes.

A new task need not start from scratch:
retrieve by node, reuse the repair.

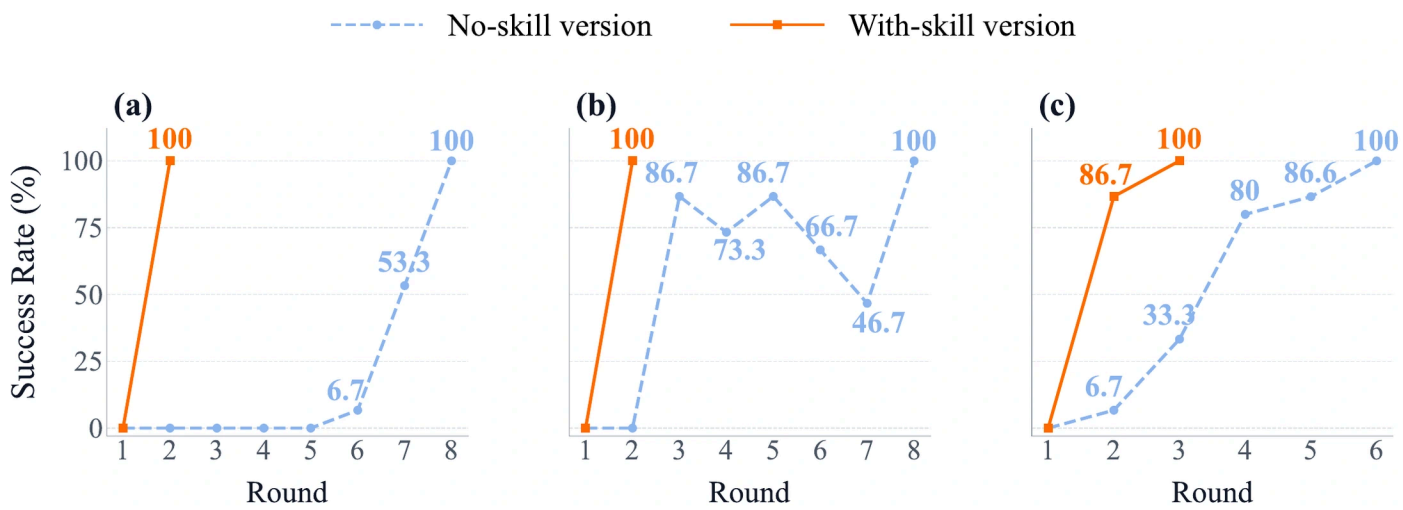


Node-level repairs transfer.
An entire new task program is not necessarily reusable unchanged.

Manuscript §4.4 Skill Library / Fig. 10 · Figure converted from the local manuscript PDF

03 · FRAMEWORK & EVOLUTION

Evolution is not always monotonic; prior skills shorten search



Original paper figure · Blue: no-skill · Orange: with-skill · x: evolution round · y: success rate

Task 0 · bottleneck removed

A sharp gain after resolving a blocking error.

Task 1 · oscillating search

Edits can introduce regressions.

Task 5 · cumulative repair

Local corrections build on one another.

Evidence With skills, these three tasks reach 100% development success within two rounds—not a universal convergence guarantee.

Manuscript §5.4 / Fig. 19 · 3 representative tasks; each round aggregates 30 development runs: seeds 0–14 × Swap / Task

EVIDENCE & TAKEAWAYS

Main results: improve success without retraining the backbone

LIBERO-PRO · $\pi_{0.5}$

+61.6 pp

Pure action model	+ HarnessPAI
34.9%	→ **96.5%**

3 suites × 2 perturbations · 60 tasks
Backbone and harness evaluated by the authors

ROBOCASA · WORLDDREAMER

+27.2 pp

Pure action model	+ HarnessPAI
65.0%	→ **92.2%**

Atomic-Seen · 18 tasks
Same WorldDreamer; unified evaluation setting

> **Protocol matters:** LIBERO-PRO's 50 evaluation seeds include the 15 evolution seeds. RoboCasa uses a fixed layout with 20 seeds per task; obstacles and distractors vary.

Manuscript §5.2 LIBERO-PRO / RoboCasa · Gains are percentage points (pp), not relative percentages

EVIDENCE & TAKEAWAYS

Backbone transfer: swap in DreamZero without re-evolution

During evolution

Code + $\pi_{0.5}$

→

Keep code, perception, geometry, and checks

Replace only the action-model calls

→

During transfer

Code + DreamZero

LIBERO-Object perturbation	Original DreamZero	Zero-shot harness transfer
Swap	0.6%	**73.4%**
Task	9.6%	**84.4%**

Implication Some system knowledge can live outside model weights and transfer to another action backend.

Not every model is a drop-in replacement. Interface compatibility and primitive competence still constrain the benefit.

Manuscript §5.3 Harness Zero-shot / Fig. 18 · LIBERO-Object Swap / Task only, not all LIBERO-PRO suites

EVIDENCE & TAKEAWAYS

A validated program can also collect expert demonstrations

Converged program
Repeat in perturbed scenes

→

Success filtering
Keep only gate-passing rollouts

→

Supervised fine-tuning
Distill experience into weights

$\pi_{0.5}$-LIBERO · three-suite mixture

+38.8 pp

LIBERO-PRO: 34.9% → 73.7%

DreamZero · LIBERO-Object

+21.4 pp

Swap / Task mean: 5.1% → 26.5%

Keep separate Main loop: frozen model, evolving code. Extension: train on successful trajectories—not alternating harness–model co-evolution.

Fine-tuning uses seeds 0–39 for training and 40–49 for testing, a different protocol from the 50-seed harness evaluation.

Manuscript §4.5 Model Feedback; §5.6 VLA/WAM Post-Training · A separate experiment from the main harness-evolution loop

EVIDENCE & TAKEAWAYS

Effective, not universal: where does the framework stop?

Favorable conditions

- Useful atomic model or control primitives.
- Observable, checkable intermediate stages.
- Repeated tasks that allow program reuse.
- Controlled trials during development.

Remaining boundaries

- Organization cannot create absent skills.
- Opaque long-horizon stages are harder to repair.
- Fixed code and checks are not a safety proof.
- Different protocols are not controlled ablations.

> Open questions: unseen-scene generalization, cheaper evolution, and adaptation under explicit safety constraints.

Manuscript §6 Discussion; §5.1 Experimental Setup; §4.5 Model Feedback

EVIDENCE & TAKEAWAYS

Take-home message: Physical AI is more than an action model

01 **Observation** Strong motor behavior can coexist with brittle grounding and task understanding.

02 **Motivation** Organize existing skills; prevent mistakes that physical execution cannot undo.

03 **Method** Execute fixed programs, evolve between rollouts, and reuse node-level repairs.

$$\text{Physical AI} \neq \text{Action Model}$$

$$\text{Physical AI} = \text{Perception} + \text{Reasoning} + \text{Action} + \text{Executable Organization}$$

Thank you · Q & A

Appendix: evaluation protocols / runtime cost / full pipeline

HarnessPAI: An Evolving Harness for Physical AI · Darwin Agent Team

APPENDIX · DISCUSSION MATERIAL

Appendix A · Read the protocol before reading the score

Experiment	Development / training	Evaluation	Interpretation
LIBERO / PRO	Evolve on seeds 0–14	50 seeds, including those 15	Not wholly unseen seeds
robosuite	Evolve on seeds 101–125	Seeds 1–100	Disjoint development / evaluation
RoboCasa Atomic	Fixed-layout evolution	18 tasks × 20 seeds	Obstacles and distractors vary
BEHAVIOR-1K	Initially evolve on 26–35	Zero-shot on seeds 1–25	Then adapt on failing test seeds
VLA / WAM fine-tuning	Seeds 0–39	Seeds 40–49	Separate from main harness tests

Reporting rule Separate development success, unseen-scene generalization, and test-scene adaptation. Cross-paper results retain their own protocols.

Manuscript §5.1 Experimental Setup; §5.4 Evolution Process; §5.6 Post-Training

APPENDIX · DISCUSSION MATERIAL

Appendix B · No online high-level reasoning is not zero cost

Mean time per rollout	Harness VLA · serial	HarnessPAI · serial	Speedup
Swap	1134.9 s	49.2 s	**23.1×**
Task	978.1 s	53.2 s	**18.4×**

What disappears at runtime

After selecting a program, execution does not require step-by-step high-level LLM deliberation.

What still costs resources

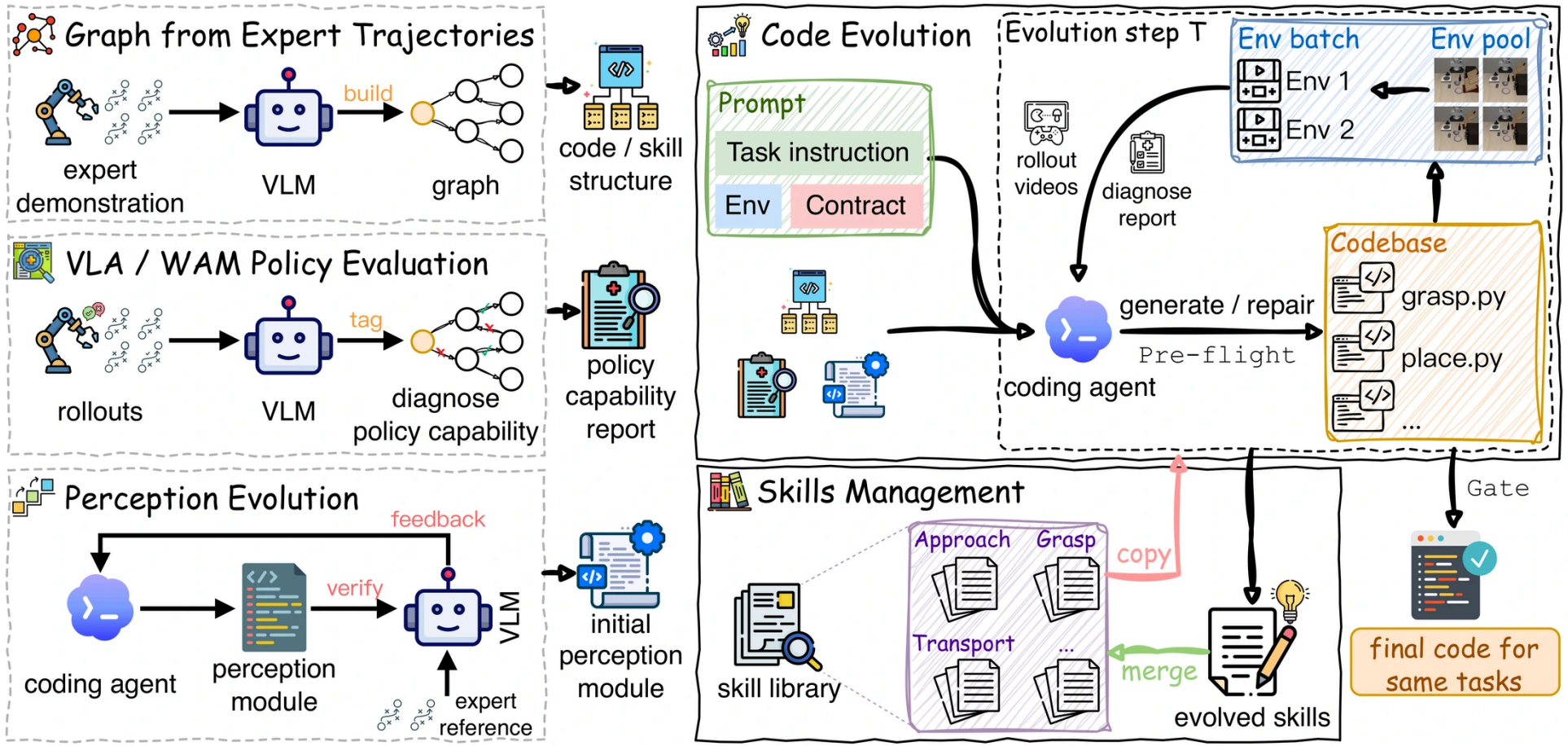
Code evolution, perception, action-model inference, control, and simulation or physical execution.

Timing covers the full execution loop. The paper separately reports a 7-GPU parallel setting; its extra speedup is not attributed solely to code here.

Manuscript §5.5 Cost / Fig. 21 · LIBERO-Goal Task 1; 50 rollouts each for Swap / Task · Serial comparison only

APPENDIX · DISCUSSION MATERIAL

Appendix C · The full evolution and memory pipeline



Initialization is optional; execution–diagnosis–repair is central. Retain both validated programs and repair experience.

Manuscript §4.2–4.4 · Original close_loop figure from the supplied project assets